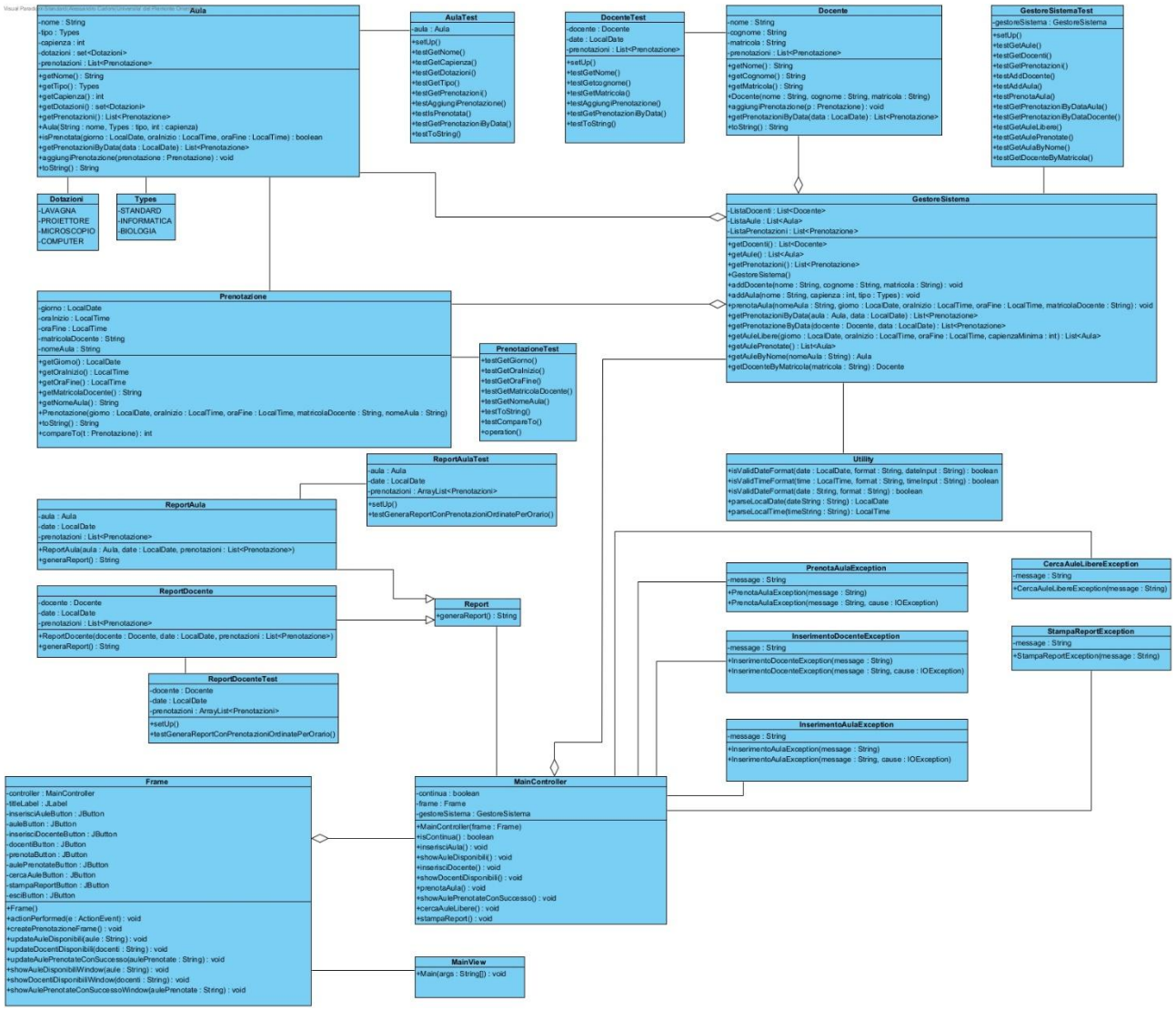
Alessandro Cadoni 20036246

Alessia Lupo 20040191

Progetto Paradigmi di Programmazione, traccia 1:



MODELLO:

- La classe **"Aula"** modella le aule di un'istituzione universitaria. Ogni aula ha un nome, una capienza massima e un tipo (standard, informatica o biologia). È dotata di specifiche attrezzature e tiene traccia delle prenotazioni. La classe offre metodi per controllare le prenotazioni e ottenere informazioni dettagliate sull'aula.
- La classe **"Docente"** modella un membro del corpo docente dell'università. Ogni docente è identificato da nome, cognome e matricola unica. La classe gestisce le prenotazioni effettuate dal docente, consentendo l'aggiunta di nuove prenotazioni e l'ottenimento di un elenco di prenotazioni

in una data specifica. Fornisce anche una rappresentazione testuale del docente con nome, cognome e matricola.

- La classe **"GestoreSistema"** gestisce il sistema dell'università. Dispone di liste per i docenti, le aule e le prenotazioni. Consente di ottenere elenchi di docenti, aule e prenotazioni. È in grado di aggiungere nuovi docenti e aule al sistema e di effettuare prenotazioni. Inoltre, fornisce metodi per ottenere aule libere e aule prenotate in base a specifici criteri. La classe offre anche la possibilità di cercare aule e docenti per nome o matricola, ignorando maiuscole e minuscole.

  Inoltre, la classe utilizza e gestisce le seguenti eccezioni:
  - InserimentoDocenteException: lanciata quando si tenta di inserire un docente già presente nella lista.
  - InserimentoAulaException: lanciata quando si tenta di inserire un'aula già esistente.
  - PrenotaAulaException: lanciata quando si tenta di prenotare un'aula già prenotata in quell'orario.
- La classe **"Prenotazione"** rappresenta una prenotazione di un'aula. Ogni prenotazione è caratterizzata da un giorno, un orario di inizio e di fine, il nome dell'aula prenotata e un identificativo del docente associato alla prenotazione. La classe offre metodi per ottenere queste informazioni e fornisce una rappresentazione testuale dettagliata della prenotazione. Inoltre, implementa l'interfaccia `Comparable` per consentire il confronto tra prenotazioni in base a data e orario.
- L'interfaccia **"Report"** definisce un contratto per la generazione di un report all'interno del sistema dell'università. Le classi che implementano "generaReport" devono fornire un'implementazione specifica per la generazione di un report sotto forma di stringa.
- La classe **"ReportAula"** rappresenta un report delle prenotazioni di un'aula in una data specifica. Il report include l'aula di riferimento, la data e un elenco delle prenotazioni ordinate per fascia oraria. Il metodo `generaReport` ordina le prenotazioni e crea una stringa formattata con le informazioni rilevanti.
- La classe **"ReportDocente"** genera un report delle prenotazioni effettuate da un docente in una data specifica. Il report contiene l'identificativo del docente, la data, e un elenco delle prenotazioni ordinate per fascia oraria. Il metodo `generaReport` ordina le prenotazioni e crea una stringa formattata con le informazioni rilevanti.
- La classe **"Utility"** fornisce metodi per validare e convertire date e ore, garantendo che siano nel formato corretto.

CONTROLLO:

- La classe **“MainController”** contiene il codice che gestisce le interazioni tra l'utente e l'applicazione. Questo include la gestione delle finestre di dialogo per inserire nuove aule e docenti, la visualizzazione delle aule e dei docenti disponibili, nonché la prenotazione di aule e la generazione di report. Viene utilizzata anche per cercare aule libere in base ai criteri specificati. La classe fa ampio uso di eccezioni personalizzate per gestire situazioni anomale durante l'esecuzione del programma.

  Inoltre, la classe utilizza e gestisce le seguenti eccezioni:
  - CercaAuleLibereException: Lanciata durante la ricerca di aule libere in caso di errori o risultati non validi.
  - PrenotaAulaException: Lanciata durante il processo di prenotazione di un'aula in caso di errori o dati non validi.
  - StampaReportException: Lanciata durante la generazione di un report in caso di errori o dati non validi.

- InserimentoAulaException: Lanciata durante l'inserimento di una nuova aula in caso di errori o dati non validi.
- InserimentoDocenteException: Lanciata durante l'inserimento di un nuovo docente in caso di errori o dati non validi.

VISTA:

- La classe **“Frame”** rappresenta l'interfaccia principale dell'applicazione per la gestione delle aule universitarie. Essa offre pulsanti per svolgere varie azioni come inserire aule, visualizzare informazioni, effettuare prenotazioni, e altro ancora. La classe gestisce gli eventi generati dagli utenti e fornisce feedback attraverso finestre di dialogo. Permette inoltre la visualizzazione di elenchi di aule, docenti e prenotazioni. L'applicazione offre diverse funzionalità, come l'inserimento di aule e docenti, la prenotazione di aule, la visualizzazione delle aule disponibili e delle prenotazioni effettuate, la ricerca di aule libere e la stampa di report.
- La classe **“MainView”** contiene il metodo principale dell'applicazione. Esso avvia l'interfaccia grafica dell'applicazione chiamando il costruttore di `Frame` e impostando un'icona per la finestra. L'applicazione viene avviata tramite il metodo “main”.

TEST:

- La classe **“AulaTest”** contiene una serie di test per la classe “Aula” dell'applicazione. I test verificano aspetti come il nome, la capienza, le dotazioni, il tipo, le prenotazioni e la rappresentazione in stringa di un'aula.
- La classe **“DocenteTest”** contiene una serie di test per la classe “Docente” dell'applicazione. I test verificano aspetti come il nome, il cognome, la matricola, l'aggiunta di prenotazioni e la rappresentazione in stringa di un docente.
- La classe **“GestoreSistemaTest”** contiene una serie di test per la classe “GestoreSistema” dell'applicazione. I test verificano aspetti come l'ottenimento di aule, docenti e prenotazioni, l'aggiunta di docenti e aule, e la prenotazione di aule. Viene inoltre testata la funzionalità di ottenere prenotazioni per data, sia per aula che per docente, e la ricerca di aule libere.
- La classe **“PrenotazioneTest”** contiene una serie di test per la classe “Prenotazione” dell'applicazione. I test verificano aspetti come l'ottenimento del giorno, dell'orario di inizio e di fine, della matricola del docente e del nome dell'aula. Viene inoltre testato il corretto utilizzo del metodo `toString()` e la funzionalità di confronto tra prenotazioni mediante il metodo `compareTo()`.
- La classe di test **"ReportAulaTest"** verifica la corretta generazione di un report delle prenotazioni per un'aula specifica nella classe "ReportAula". Il report è ordinato per orario di inizio delle prenotazioni.
- La classe di test **"ReportDocenteTest"** verifica la corretta generazione di un report delle prenotazioni per un docente specifico nella classe "ReportDocente". Il report è ordinato per orario di inizio delle prenotazioni.

MODIFICHE EFFETTUATE:

- Eliminazione della classe “GestioneAule” gestendo le dotazioni specifiche di un'aula direttamente all'interno della classe Aula, evitando così incongruenze.
- Aggiunta di una classe, in questo caso “GestoreSistema”, svolge il compito di gestire e coordinare il sistema dell'università.
- È stata eliminata la classe “PrenotazioniSuccesso”, ritenuta superflua per l'applicazione in quanto presentava ridondanze. L'attributo 'nomeAula' è stato incorporato nella classe “Prenotazione”.

- Il report è stato reso un'interfaccia implementata da ReportAula e ReportDocente.
- Suddivisione del pattern MVC, con strutturazione di un modello, un controllore e una vista.

TABELLA:

Test per le classi del modello	Fatto
Javadoc per i componenti pubblici del modello	Fatto
Il modello non fa alcuna lettura/scrittura da console o altra interazione con l'utente	Fatto
È stata usata l'ereditarietà ovunque appropriato	Fatto
È stato riutilizzato il codice ovunque appropriato	Fatto
Sono state utilizzate visibilità appropriate	Fatto
Sono state lanciate eccezioni ove appropriato	Fatto
Sono state usate classi annidate ove appropriato	Fatto
Specificare dove/come è stata fatta l'estensione di una classe o classe astratta	Uso e implementazione di un'interfaccia.